DOCUMENT

Metadata Analyzer — dokumentacja techniczna etapu 1

Dokumentacja techniczna wersji 1.1 (2026-11-11) opisuje architekturę, warstwy Core, Infrastructure oraz interfejsy WPF i CLI działającego szkieletu aplikacji Metadata Analyzer na platformie .NET 10 (LTS).

11 November 2026

1. Cel i zakres etapu oraz wymagania i kryteria akceptacji

Strona tytułowa

Tytuł	Metadata Analyzer — dokumentacja techniczna etapu 1
Wersja dokumentu	1.1
Data	2026-11-11
Wersja aplikacji	1.0.0
Platforma	.NET 10 (LTS), WPF, Windows
Stan jakości	build 0 błędów / 0 ostrzeżeń, 54 z 54 testów zielonych
Źródło treści	`DOKUMENTACJA.md` oraz wersja PDF dokumentacji

Cel i zakres etapu

Celem etapu 1 jest **działający szkielet** aplikacji Metadata Analyzer: dwa niezależne interfejsy (WPF i CLI), wspólna logika, wspólne modele danych, rejestrowanie przebiegu działania w pliku tekstowym oraz testy jednostkowe najważniejszych funkcji.

Etap 1 **nie** zawiera prawdziwej analizy metadanych przez `System.Reflection` ani bazy danych — dane pochodzą z dostawcy symulowanego, a logger zapisuje wyłącznie do pliku. Oba elementy są przygotowane do wymiany w kolejnych etapach dzięki interfejsom i wstrzykiwaniu zależności.

Zakres dostarczony:

- rozwiązanie z 5 projektami (`Core`, `Infrastructure`, `WPF`, `CLI`, `Tests`),
- pełna hierarchia modeli metadanych,
- dostawca danych symulowanych `SampleLibrary.dll`,
- serwis logiki `MetadataService` + formatter prezentacji tekstowej,
- logger plikowy z poziomami INFO/WARNING/ERROR,
- okno WPF w MVVM (drzewo, panel szczegółów, pasek statusu),
- aplikacja konsolowa z menu i obsługą błędnych danych wejściowych,
- 54 testy jednostkowe (xUnit + Moq),
- konfiguracja `appsettings.json` i kompozycja DI.

Wymagania i kryteria akceptacji

#	Wymaganie etapu	Status	Realizacja / dowód
1	Uruchomienie GUI WPF	✅	`dotnet run --project MetadataAnalyzer.WPF`; zrzut `docs/wpf-preview.png`

#	Wymaganie etapu	Status	Realizacja / dowód
2	Niezależne uruchomienie CLI	✅	`dotnet run --project MetadataAnalyzer.CLI` ; zapis `docs/cli-smoke.txt`
3	Wygenerowanie danych symulowanych	✅	`MockMetadataProvider` (3 przestrzenie nazw, 4 klasy, 14 składników)
4	Hierarchiczne drzewo w WPF	✅	`TreeNodeViewModel` + `TreeView` z szablonem hierarchicznym
5	Ta sama struktura w konsoli	✅	`MetadataFormatter.FormatTree` — wcięcia `| +--`
6	Szczegóły wybranego elementu	✅	`DetailsViewModel` + `DetailsView` ; brak przeładowania okna
7	Rejestrowanie przebiegu w pliku	✅	`FileLogger` → `Logs/application.log`
8	Testy jednostkowe	✅	54 testy, 0 niepowodzeń

Kryterium zaliczenia (WPF): wybranie klasy w drzewie powoduje aktualizację panelu szczegółów bez przeładowania okna — spełnione: `TreeView.SelectedItemChanged` → `MainViewModel.SelectedNode` → `DetailsViewModel` → `ContentControl` w panelu prawym. Drzewo i model nie są przebudowywane.

Kryterium zaliczenia (CLI): ten sam `IMetadataService` co WPF — spełnione: oba interfejsy rozwiązują `IMetadataService` z kontenera DI i nie zawierają własnej logiki analizy ani generowania danych.

2. Architektura rozwiązania

2.1. Podział na projekty

```
MetadataAnalyzer/
├── MetadataAnalyzer.slnx
├── Directory.Build.props
│
├── MetadataAnalyzer.Core/
│   ├── Models/          AssemblyModel, NamespaceModel, ClassModel, MethodModel,
│   │                    PropertyModel, FieldModel, ConstructorModel, ParameterModel,
│   │                    MemberVisibility
│   ├── Interfaces/      IMetadataProvider, IMetadataService, IMetadataFormatter,
│   │                    IAppLogger, ISystemClock
│   └── Services/        MockMetadataProvider, MetadataService, MetadataFormatter,
│                        MetadataProviderResolver, SystemClock, LogEvents,
│                        ServiceCollectionExtensions
│
├── MetadataAnalyzer.Infrastructure/
│   ├── Configuration/   ApplicationOptions
│   ├── Logging/         FileLogger, LoggingOptions, ServiceCollectionExtensions
│   └── AppConfiguration.cs
│
├── MetadataAnalyzer.WPF/
│   ├── Views/           MainWindow, DetailsView
│   ├── ViewModels/      MainViewModel, AssemblyViewModel, TreeNodeViewModel,
│   │                    DetailsViewModel, TreeNodeKind
│   ├── Converters/      BoolToVisibilityConverter, CountToVisibilityConverter,
│   │                    NullToVisibilityConverter
│   ├── Themes/          Colors.xaml, Controls.xaml
│   ├── App.xaml / App.xaml.cs
│   ├── appsettings.json, app.manifest
│
├── MetadataAnalyzer.CLI/
│   ├── Program.cs, MetadataConsoleApp.cs, ConsolePresentation.cs, appsettings.json
│
└── MetadataAnalyzer.Tests/
    ├── Models/ModelTests.cs
    ├── Services/  MockMetadataProviderTests, MetadataServiceTests,
MetadataFormatterTests
    ├── Logging/   FileLoggerTests.cs
    ├── ViewModels/ MainViewModelTests, TreeNodeViewModelTests
    └── Support/   Fakes.cs (FakeClock, RecordingLogger, TemporaryDirectory)
```

Poza pięcioma projektami rozwiązanie zawiera pliki poziomu repozytorium: `MetadataAnalyzer.slnx` (definicja rozwiązania), `Directory.Build.props` (wspólne ustawienia kompilacji), `README.md`, `DOKUMENTACJA.md` (wersja jednoplikowa dokumentacji) oraz katalog `docs/` z materiałami towarzyszącymi (zrzut WPF, zapis sesji CLI, wersja PDF dokumentacji).

2.2. Graf zależności

```
MetadataAnalyzer.WPF ──────┐
                           ├──> MetadataAnalyzer.Core
MetadataAnalyzer.CLI ──────┘

MetadataAnalyzer.Infrastructure ──> MetadataAnalyzer.Core

MetadataAnalyzer.Tests ──> Core + Infrastructure + WPF
```

`Core` nie ma odwołań do WPF, CLI ani do konkretnej implementacji logowania. Zawiera wyłącznie modele POCO, interfejsy i logikę aplikacji. Jedynym pakietem w `Core` są abstrakcje kontenera DI (`Microsoft.Extensions.DependencyInjection.Abstractions`) — bez implementacji kontenera.

2.3. Przepływ danych

```
MockMetadataProvider
        │  AssemblyModel
        ▼
  MetadataService  ──(IAppLogger)──>  FileLogger  ──>  Logs/application.log
        │  AssemblyModel
    ┌───┴────┐
    ▼        ▼
  WPF       CLI
   │         │
TreeNodeVM   MetadataFormatter.FormatTree
DetailsVM    MetadataFormatter.FormatDetails
```

Kluczowa zasada: **żaden interfejs nie generuje danych i nie implementuje analizy**. WPF i CLI jedynie prezentują to, co zwróci `IMetadataService`, oraz formatują model do tekstu/logiki widoku.

3. Modele danych Core

Wszystkie modele to niemutowalne `record` (wartościowa równość, wygodne testowanie), a kolekcje typu `IReadOnlyList<T>` są normalizowane w konstruktorze — nigdy nie są `null`.

3.1. `AssemblyModel`

```
public sealed record AssemblyModel(
    string Name,
    string Version,
    string FileName,
    IReadOnlyList<NamespaceModel>? Namespaces = null,
    DateTimeOffset LoadedAt = default)
```

Składowa	Typ	Znaczenie
`Name`	`string`	Nazwa zestawu (`SampleLibrary`)
`Version`	`string`	Wersja (`1.0.0.0`)
`FileName`	`string`	Nazwa pliku (`SampleLibrary.dll`)
`Namespaces`	`IReadOnlyList<NamespaceModel>`	Przestrzenie nazw (domyślnie pusta lista)
`LoadedAt`	`DateTimeOffset`	Moment analizy (z `ISystemClock`)
`Classes`	`IEnumerable<ClassModel>`	Spłaszczone klasy ze wszystkich przestrzeni
`NamespaceCount`	`int`	Liczba przestrzeni nazw
`ClassCount`	`int`	Liczba klas
`MemberCount`	`int`	Łączna liczba składników (pola + właściwości + konstruktory + metody)
`Empty(name)`	`static`	Fabryka pustego modelu — używana w testach scenariuszy brzegowych

3.2. `NamespaceModel`

```
public sealed record NamespaceModel(string Name, IReadOnlyList<ClassModel>? Classes =
null)
```

`Name` to pełna nazwa przestrzeni (`SampleLibrary.Models`), `Classes` — klasy w przestrzeni.

3.3. `ClassModel`

```
public sealed record ClassModel(
    string Name, string FullName, string? BaseType = null,
    IReadOnlyList<MethodModel>? Methods = null,
    IReadOnlyList<PropertyModel>? Properties = null,
    IReadOnlyList<FieldModel>? Fields = null,
    IReadOnlyList<ConstructorModel>? Constructors = null,
    MemberVisibility Visibility = MemberVisibility.Public,
    bool IsStatic = false, bool IsAbstract = false, bool IsInterface = false)
```

Dodatkowo: `DisplayName` (dla interfejsów poprzedza nazwę słowem `interface`) oraz `MemberCount` (suma liczby metod, właściwości, pól i konstruktorów). Zestaw właściwości pokrywa wymagania etapu i rozszerza je o informacje potrzebne w panelu szczegółów (`Visibility` , `IsStatic` , `IsAbstract` , `IsInterface`).

3.4. Składniki klasy

Model	Właściwości	Uwagi
`MethodModel`	`Name`, `ReturnType`, `Visibility`, `IsStatic`, `Parameters`	`Signature` → `GetUser(int id) : User` (bez typu zwracanego)
`PropertyModel`	`Name`, `PropertyType`, `CanRead`, `CanWrite`, `Visibility`, `IsStatic`	`CanRead` / `CanWrite` opisują akcesory
`FieldModel`	`Name`, `FieldType`, `Visibility`, `IsStatic`, `IsReadOnly`	pola prywatne widoczne w drzewie
`ConstructorModel`	`Visibility`, `Parameters`	`Signature` → `(IUserRepository userRepository)`
`ParameterModel`	`Name`, `ParameterType`, `IsOptional`	`DisplayName` → `User fallback = default` dla opcjonalnego
`MemberVisibility`	`enum`	`Public, Protected, Internal, Private, ProtectedInternal, PrivateProtected`

`MethodModel.Signature` celowo nie zawiera typu zwracanego — formatter dokłada go osobno (`Method: GetFullName() : string`), dzięki czemu podpis jest czytelny w drzewie i w konsoli.

3.5. Odwzorowanie zagnieżdżeń

```
AssemblyModel (1) ──> NamespaceModel (*) ──> ClassModel (*)
                                              ├── MethodModel (*) ──> ParameterModel (*)
                                              ├── PropertyModel (*)
                                              ├── FieldModel (*)
                                              └── ConstructorModel (*) ──> ParameterModel (*)
```

Typy zwracane i typy parametrów są przechowywane jako nazwy (`"User"`, `"int"`, `"IUserRepository"`), co pozwala odwoływać się do typów spoza analizowanego zestawu bez cyklicznych referencji obiektowych. W etapie 2, przy analizie przez `System.Reflection`, te nazwy będą uzupełniane o informację, czy typ należy do analizowanego zestawu.

3.6. `List<T>` czy `ObservableCollection<T>`

Modele w `Core` używają `IReadOnlyList<T>`, ponieważ dane przekazywane są jako **gotowy, niezmienny model**. Kolekcja obserwowana (`ObservableCollection<TreeNodeViewModel>`) występuje wyłącznie w warstwie WPF — tam, gdzie drzewo jest bezpośrednio obserwowane przez kontrolkę `TreeView`.

4. Interfejsy Core

```
 public interface IMetadataProvider
{
    Task<AssemblyModel> LoadAsync(CancellationToken cancellationToken = default);
}

public interface IMetadataService
{
    AssemblyModel? Current { get; }
    Task<AssemblyModel> AnalyzeAsync(CancellationToken cancellationToken = default);
    Task<AssemblyModel> RefreshAsync(CancellationToken cancellationToken = default);
}

public interface IMetadataFormatter
{
    string FormatTree(AssemblyModel assembly);
    string FormatDetails(AssemblyModel assembly);
    string FormatMemberDetails(ClassModel type, string memberName);
}

public interface IAppLogger
{
    Task LogInformationAsync(string message);
    Task LogWarningAsync(string message);
    Task LogErrorAsync(string message);
}

public interface ISystemClock
{
    DateTimeOffset Now { get; }
}
```

Interfejs	Rola	Implementacje
`IMetadataProvider`	źródło metadanych (dziś mock, w etapie 2 refleksja)	`MockMetadataProvider`, `MetadataProviderResolver`
`IMetadataService`	logika aplikacji: analiza, odświeżanie, bieżący model	`MetadataService`
`IMetadataFormatter`	formatowanie modelu do tekstu (CLI, logi, eksport)	`MetadataFormatter`
`IAppLogger`	rejestrowanie zdarzeń	`FileLogger`, `RecordingLogger` (testy)
`ISystemClock`	źródło czasu (testowalność `LoadedAt` i pomiaru czasu operacji)	`SystemClock`, `FakeClock` (testy)

Parametr `ClassModel type` w `FormatMemberDetails` (a nie `@class`) to świadoma decyzja zgodna z regułą CA1716 — unikanie słów kluczowych w nazwach parametrów publicznych API.

5. Usługi Core

5.1. `MockMetadataProvider`

Buduje pełne drzewo `SampleLibrary.dll` w metodzie `LoadAsync`. Dane tworzone są **w providerze, a nie w ViewModelu** — zgodnie z wymaganiem etapu. Metoda `BuildAssembly(DateTimeOffset)` jest publiczna i statyczna, dzięki czemu można jej użyć w testach bez zależności od zegara.

Czas `LoadedAt` pochodzi z `ISystemClock` (wstrzyknięty przez konstruktor, opcjonalny):

```
public sealed class MockMetadataProvider(ISystemClock? clock = null) :
IMetadataProvider
```

5.2. `MetadataService`

Odpowiedzialności:

- uruchomienie analizy (`AnalyzeAsync`) i odświeżenia (`RefreshAsync`),
- wywołanie providera i zapamiętanie wyniku w `Current` ,
- logowanie startu, sukcesu, anulowania i błędu,
- serializacja równoległych wywołań (`SemaphoreSlim` , `IDisposable`),
- propagowanie wyjątku do interfejsu po zalogowaniu.

Przepływ `LoadAsync` :

```
LogInformationAsync(startMessage)
  └─ provider.LoadAsync(token)
       └─ Current = assembly
            └─ LogInformationAsync("Sample data loaded: …")
  wyjątek? → LogErrorAsync("Metadata analysis failed: …") → rethrow
  anulowanie? → LogWarningAsync("Metadata loading cancelled") → rethrow
  finally → _gate.Release()
```

5.3. `MetadataProviderResolver`

Cienka warstwa decyzyjna pozwalająca zachować kontrakt `UseMockData` z konfiguracji bez komplikowania kompozycji DI. Gdy `UseMockData = false` (etap 2, jeszcze niezaimplementowany), rzuca `NotSupportedException` z czytelnym komunikatem zamiast po cichu zwracać dane mockowe.

5.4. `MetadataFormatter`

Trzy operacje formatowania tekstu:

Metoda	Wynik	Użycie
`FormatTree`	drzewo ASCII z wcięciami	CLI (opcja 2), logi, eksport

Metoda	Wynik	Użycie
`FormatDetails`	podsumowanie zestawu + lista przestrzeni i klas	CLI (opcja 3)
`FormatMemberDetails`	szczegóły jednego składnika klasy	CLI (opcja 4)

Przykład `FormatTree`:

```
 Assembly: SampleLibrary
Version: 1.0.0.0

|   +-- Namespace: SampleLibrary.Models
    |   +-- Class: User
        |   +-- Property: Id (int)
        |   +-- Property: Name (string)
        |   +-- Constructor: public ()
        +-- Method: GetFullName() : string
    +-- Class: Product
        |   +-- Property: Id (int)
        |   +-- Property: Price (decimal)
        |   +-- Constructor: public ()
        +-- Method: CalculateTax() : decimal
|   +-- Namespace: SampleLibrary.Services
```

Szczegóły implementacyjne:

- wcięcie to powtórzone `" "` na każdy poziom (`GetIndent`), connector `+--` lub `| +--`,
- kolejność składników jest deterministyczna: pola → właściwości → konstruktory → metody,
- całe formatowanie używa `CultureInfo.InvariantCulture` (reguła CA1305 jako błąd kompilacji),
- puste modele (`AssemblyModel.Empty()`) obsłużone jawnie (`"Assembly contains no namespaces."`),
- nieznany składnik → `"Member: Missing (not found)"` zamiast wyjątku.

5.5. `LogEvents`

Centralny słownik komunikatów logu — jedna zmiana nazwy zdarzenia w jednym miejscu:

```
 public const string ApplicationStarted   = "Application started";
public const string ApplicationClosed    = "Application closed";
public const string MetadataAnalysisStarted = "Metadata analysis started";
public const string SampleDataLoaded     = "Sample data loaded";
public const string MetadataDisplayed    = "Metadata displayed";
public const string DetailsDisplayed     = "Details displayed";
public const string MetadataAnalysisFailed = "Metadata analysis failed";
public const string InvalidSelection     = "Invalid selection";
```

6. Dane symulowane SampleLibrary.dll

Dane symulowane dostarcza `SampleLibrary.dll` w wersji `1.0.0.0`:

```
SampleLibrary.dll
│
├── SampleLibrary.Models
│   ├── User
│   │   ├── Property Id : int
│   │   ├── Property Name : string
│   │   ├── Constructor public ()
│   │   └── Method GetFullName() : string
│   └── Product
│       ├── Property Id : int
│       ├── Property Price : decimal
│       ├── Constructor public ()
│       └── Method CalculateTax() : decimal
│
├── SampleLibrary.Services
│   └── UserService
│       ├── Field _userRepository : IUserRepository (private, readonly)
│       ├── Constructor public (IUserRepository userRepository)
│       ├── Method GetUser(int id) : User
│       └── Method DeleteUser(int id) : bool
│
└── SampleLibrary.Helpers
    └── Logger
        ├── Constructor private ()
        └── Method static Write(string message) : void
```

Statystyki modelu: **3 przestrzenie nazw, 4 klasy, 14 składników**.

Dane pokrywają wszystkie rodzaje składników (pole, właściwość, konstruktor, metoda, parametr), wszystkie poziomy widoczności (public, private) oraz przypadek metody statycznej — dzięki temu każda ścieżka prezentacji jest widoczna w demo i testach.

7. Logowanie (FileLogger)

7.1. Format wpisu

```
[2026-11-10 20:27:49] INFO  Application started
[2026-11-10 20:27:49] INFO  Metadata analysis started
[2026-11-10 20:27:49] INFO  Sample data loaded: SampleLibrary (SampleLibrary.dll)
[2026-11-10 20:27:49] WARN  Invalid selection: '7'
[2026-11-10 20:27:49] INFO  Application closed
```

`[znacznik czasu] POZIOM(tab. 5 znaków) treść`; czas lokalny, format `yyyy-MM-dd HH:mm:ss`, kultura niezmienna. Poziomy: `INFO`, `WARN`, `ERROR`.

7.2. Wymagania i realizacja

Wymaganie	Realizacja
Poziomy INFO / WARNING / ERROR	`LogInformationAsync`, `LogWarningAsync`, `LogErrorAsync`
Data i godzina w wpisie	`DateTimeOffset.Now` + format niezmienny
Dopisywanie, nie nadpisywanie	`FileStream(FileMode.Append, FileAccess.Write, FileShare.ReadWrite)`
Obsługa błędów zapisu	`catch` na `IOException`, `UnauthorizedAccessException`, `NotSupportedException` → komunikat `[logging failure]`, **bez wyjątku na zewnątrz**
Wstrzykiwanie przez DI	rejestracja `IAppLogger → FileLogger` w `AddMetadataAnalyzerLogging`
Katalog pliku	tworzony automatycznie (`Directory.CreateDirectory`)
Ścieżka	względna rozwiązywana od `AppContext.BaseDirectory` (`AppConfiguration.ResolveLogFilePath`)
Bezpieczeństwo wątków	`SemaphoreSlim(1,1)` — wpisy nie przeplatają się; kolejność wywołań zachowana
Możliwość wyłączenia	`Logging:Enabled = false`

7.3. Zachowanie przy awarii zapisu

Logger nigdy nie przerywa działania aplikacji. Gdy zapis do pliku jest niemożliwy, próbuje poinformować o tym konsolę (`[logging failure] …`), a wywołanie kończy się sukcesem. Test `LogWriteFailureDoesNotThrow` weryfikuje ten scenariusz, wskazując jako cel plik katalogu (konstrukcja `path/application.log`, gdzie `path` jest plikiem).

7.4. Logowanie do konsoli

`LoggingOptions.LogToConsole` domyślnie `true`, ale **oba interfejsy wyłączają je** (`logToConsole: false`), aby log nie zaśmiecał prezentacji CLI i nie był bezużyteczny w WPF. Można je włączyć jednym parametrem — przydatne przy diagnostyce.

8. Konfiguracja i wstrzykiwanie zależności

8.1. `appsettings.json`

Plik jest kopiowany do katalogu wyjściowego obu aplikacji:

```
 {
   "Application": { "Name": "Metadata Analyzer", "Version": "1.0.0", "UseMockData":
true },
   "Logging": { "Enabled": true, "Provider": "File", "FilePath":
"Logs/application.log" }
}
```

Klucz	Typ	Domyślna	Znaczenie
`Application:Name`	string	Metadata Analyzer	nazwa aplikacji (WPF: tytuł okna)
`Application:Version`	string	1.0.0	wersja prezentowana w UI
`Application:UseMockData`	bool	true	`true` = dane symulowane; `false` zarezerwowane dla etapu 2
`Logging:Enabled`	bool	true	wyłączenie logowania bez zmiany kodu
`Logging:Provider`	string	File	rezerwa pod logger bazodanowy (etap 4)
`Logging:FilePath`	string	Logs/application.log	ścieżka względna (od katalogu aplikacji) lub bezwzględna

8.2. Kompozycja

Jeden punkt wejścia dla obu interfejsów:

```
 services.AddMetadataAnalyzer(configuration, logToConsole: false);
```

Wewnętrznie:

```
 services.AddSingleton<ISystemClock, SystemClock>();
services.AddSingleton<IMetadataFormatter, MetadataFormatter>();
services.AddSingleton<MockMetadataProvider>();
services.AddSingleton(new MetadataProviderOptions { UseMockData =
application.UseMockData });
services.AddSingleton<IMetadataProvider, MetadataProviderResolver>();
services.AddSingleton<IMetadataService, MetadataService>();
services.AddSingleton<ApplicationOptions>(application);
services.AddSingleton(loggingOptions);
services.AddSingleton<IAppLogger>(_ ⇒ new FileLogger(loggingOptions));
```

Rejestracja	Cykl życia	Uzasadnienie
`ISystemClock`	singleton	bezstanowe źródło czasu
`IMetadataFormatter`	singleton	bezstanowe, czyste funkcje
`MockMetadataProvider`	singleton	tanie, bezstanowe
`IMetadataProvider`	singleton (resolver)	jedno miejsce decyzji o źródle danych

Rejestracja	Cykl życia	Uzasadnienie
`IMetadataService`	singleton	przechowuje `Current` i serializuje analizę
`IAppLogger`	singleton	jeden uchwyt pliku, jedna kolejka wpisów

8.3. Wymiana providera w etapie 2

```
services.AddSingleton<IMetadataProvider, ReflectionMetadataProvider>();
```

Reszta kodu pozostaje bez zmian — serwis, formatter, ViewModele i CLI zależą wyłącznie od interfejsu.

9. Interfejs WPF (MVVM)

9.1. Układ okna

```
┌──────────────────────────────────────────────────────────────────────┐
│ [MA]  Metadata Analyzer                    [ Analyze ]  [ Refresh ]  │  pasek górny
│       SampleLibrary.dll · v1.0.0.0                                   │
├──────────────────────────────┬───────────────────────────────────────┤
│ STRUCTURE   3 ns · 4 cls …   │ [Assembly]                            │
│                              │ SampleLibrary                         │  panel szczegółów
│ ▾ SampleLibrary   Assembly   │ SampleLibrary.dll                     │
│   ▸ SampleLibrary.Models     │ ──────────────────────────────────    │
│   ▸ SampleLibrary.Services   │ Version   1.0.0.0                     │
│   ▸ SampleLibrary.Helpers    │ Namespaces 3    Classes 4   Members 14│
│                              │ Namespaces: Models / Services / Helpers│
├──────────────────────────────┴───────────────────────────────────────┤
│ ● Analysis completed in 22 ms      [Namespaces 3][Classes 4][Members 14] │  pasek statusu
└──────────────────────────────────────────────────────────────────────┘
```

9.2. ViewModele

ViewModel	Odpowiedzialność	Kluczowe składowe
`MainViewModel`	stan okna, polecenia, status	`Assembly`, `SelectedNode`, `Details`, `StatusMessage`, `IsBusy`, `AnalyzeCommand`, `RefreshCommand`, `ClearSelectionCommand`
`AssemblyViewModel`	dane analizowanego komponentu	`Name`, `Version`, `FileName`, `LoadedAt`, `Nodes`, `NamespaceCount`, `ClassCount`, `MemberCount`, `Summary`, `Counters`
`TreeNodeViewModel`	jeden węzeł drzewa	`Label`, `Caption`, `Kind`, `Model`, `Children`, `HasChildren`, `IsExpanded`, `IsSelected`, `Owner`, `ToDetails()`

ViewModel	Odpowiedzialność	Kluczowe składowe
`DetailsViewModel`	szczegóły wybranego elementu	`Title`, `Subtitle`, `Kind`, `Properties`, `Sections`, `HasSelection` + fabryki `FromAssembly`, `FromNamespace`, `FromClass`, `FromMethod`, `FromProperty`, `FromField`, `FromConstructor`, `FromParameter`

`DetailsViewModel` jest niemutowalny — zmiana wyboru podmienia cały obiekt (`Details = value.ToDetails()`), co upraszcza powiadomienia (`[ObservableProperty]` na `MainViewModel`) i eliminuje ręczne synchronizowanie dziesiątek właściwości.

9.3. Polecenia

`CommunityToolkit.Mvvm` generuje implementacje `ICommand` z atrybutów:

```
 [RelayCommand]
private async Task AnalyzeAsync(CancellationToken cancellationToken) { … }

[RelayCommand]
private async Task RefreshAsync(CancellationToken cancellationToken) { … }

[RelayCommand]
private void ClearSelection() { … }
```

Generatory tworzą `AnalyzeCommand`, `RefreshCommand`, `ClearSelectionCommand` (typ `AsyncRelayCommand` dla metod asynchronicznych — z blokadą ponownego wykonania w trakcie pracy i przekazywaniem `CancellationToken`). Dzięki temu w projekcie nie ma ręcznie pisanych `RelayCommand` / `AsyncRelayCommand` ani powtarzalnego `INotifyPropertyChanged`.

9.4. Powiadamianie o zmianach

- `[ObservableProperty]` na polach `_assembly`, `_selectedNode`, `_details`, `_statusMessage`, `_isBusy`,
- `partial void OnSelectedNodeChanged(TreeNodeViewModel? value)` — aktualizacja `Details` i statusu,
- `SetProperty` w `TreeNodeViewModel` dla `IsExpanded` / `IsSelected`,
- `OnPropertyChanged(nameof(HasMetadata))` i `OnPropertyChanged(nameof(Subtitle))` po wczytaniu modelu.

9.5. Code-behind

`MainWindow.xaml.cs` zawiera wyłącznie:

1. wstrzyknięcie `MainViewModel` przez konstruktor i przypisanie `DataContext`,
2. obsługę `SelectedItemChanged` (jedno zdarzenie, którego `TreeView` nie da się zbindować w XAML),

3. synchronizację widoczności pustego stanu z `HasMetadata` (obejście ograniczenia: w szablonie drzewa nie można wiązać z zasobami aplikacji — szczegóły w rozdz. 13).

Żadnej logiki biznesowej w widoku.

9.6. Styl i wygląd

- `Themes/Colors.xaml` — paleta (tła `#10151C` / `#151B24`, akcent `#4C8DFF`, tekst `#E6EDF5` / `#9AABBD`),
- `Themes/Controls.xaml` — style kontrolek: karty (`SurfaceCardStyle`), przyciski (`PrimaryButtonStyle`, `SecondaryButtonStyle`), drzewo (`TreeViewStyle`, `TreeViewItemStyle`), pasek przewijania, odznaki,
- typografia: `Segoe UI Variable Text` dla UI, `Cascadia Mono` dla nazw typów i podpisów,
- kolorowe kropki przy węzłach (zestaw / przestrzeń / klasa) ułatwiają orientację w hierarchii,
- drzewo bez ramek, z zaokrąglonym podświetleniem wiersza; ikona rozwinięcia to obracający się trójkąt.

9.7. Zachowanie interakcji

Akcja	Efekt
`Analyze`	ładuje dane mockowe, przebudowuje drzewo, pokazuje szczegóły zestawu, status `Analysis completed in N ms`
`Refresh`	to samo przez `IMetadataService.RefreshAsync` (osobna ścieżka logowania)
Klik w węzeł	`Details` aktualizowane natychmiast, drzewo nietknięte
Rozwinięcie węzła	`IsExpanded` w ViewModelu (stan zachowany przy przewijaniu)
Błąd analizy	status `Analysis failed: …`, log `ERROR`, okno pozostaje sprawne
Praca w toku	status `… in progress…`, wskaźnik w pasku statusu zmienia kolor; kolejne wywołanie ignorowane (`IsBusy`)

10. Interfejs CLI

10.1. Menu

```
 ========================================
           METADATA ANALYZER
 ========================================

 [1] Load simulated metadata
 [2] Display assembly structure
 [3] Display assembly details
 [4] Display member details
 [0] Exit

 State: Ready (no metadata loaded) | Namespaces: 0 | Classes: 0 | Last operation: -
 Select option:
```

Opcja	Działanie	Log
`1`	`IMetadataService.AnalyzeAsync` + podsumowanie	INFO ×2
`2`	`FormatTree` + `Analysis completed successfully.`	—
`3`	`FormatDetails` (liczniki, lista przestrzeni i klas)	—
`4`	wybór klasy (lista numerowana) → wybór składnika → `FormatMemberDetails`	—
`0`	wyjście (`Goodbye.`)	INFO `Application closed`
inna	`Invalid option 'X'. Choose one of: 1, 2, 3, 4, 0.`	WARN `Invalid selection: 'X'`

10.2. Odporność na błędy wejścia

- opcja spoza zakresu → ostrzeżenie + log WARN, powrót do menu,
- `2` / `3` / `4` bez wczytanych danych → `No metadata loaded. Select option 1 first.`,
- numer klasy/składnika spoza zakresu → `Invalid number. Enter a value between 1 and N.`,
- koniec strumienia wejścia (EOF, np. przy przekierowaniu) → czyste wyjście z kodu `0`,
- `Ctrl+C` → `CancellationToken` → `Operation cancelled.` i kod wyjścia `1`,
- nieoczekiwany wyjątek → log ERROR + komunikat na `stderr` + kod wyjścia `1`.

10.3. Struktura kodu

Klasa	Odpowiedzialność
`Program`	konfiguracja, DI, obsługa `Ctrl+C`, kody wyjścia
`MetadataConsoleApp`	pętla menu, obsługa opcji, wybór klasy i składnika
`ConsolePresentation`	cała warstwa wizualna: nagłówek, stopka, menu, kolory, etykiety, tabela wyboru

Rozdzielenie prezentacji od logiki sterowania ułatwia testowanie (wejście/wyjście jako `TextReader` / `TextWriter` wstrzykiwane przez DI) i późniejszą wymianę na bogatszy interfejs.

11. Testy jednostkowe

11.1. Narzędzia i organizacja

- **xUnit 2.9.3**, **Moq 4.21.0**, `Microsoft.NET.Test.Sdk`, `coverlet.collector`,
- projekt `net10.0-windows` (testuje też ViewModele WPF),
- struktura katalogów odpowiada warstwom: `Models/`, `Services/`, `Logging/`, `ViewModels/`, `Support/`,
- `Support/Fakes.cs` — `FakeClock` (sterowalny zegar), `RecordingLogger` (zapis wpisów w listach), `TemporaryDirectory` (izolowany katalog z automatycznym sprzątaniem).

11.2. Wynik

```
Powodzenie! — niepowodzenie: 0, powodzenie: 54, pominięto: 0, łącznie: 54
```

11.3. Zakres pokrycia

Obszar	Testy	Sprawdzane zachowania
`ModelTests`	7	puste modele, agregacja liczników, `MemberCount`, podpisy metod i konstruktorów, domyślne puste kolekcje, równość wartościowa
`MockMetadataProviderTests`	6	nazwa/wersja/plik, struktura 3 ns / 4 klasy, wymagane klasy i składniki, `LoadedAt` z zegara, świeże instancje, anulowanie
`MetadataServiceTests`	7	model z providera, logi start/sukces, błąd + rethrow + `Current == null`, anulowanie + WARN, przekazanie `CancellationToken`, `RefreshAsync` podmienia model, `Current` puste przed analizą
`MetadataFormatterTests`	10	nagłówek, hierarchia i wcięcia, pusty model, `null` → `ArgumentNullException`, liczniki, metoda/właściwość/nieznany składnik, pusty `memberName`, niezależność od kultury (pl-PL vs invariant)

Obszar	Testy	Sprawdzane zachowania
`FileLoggerTests`	9	zapis INFO/WARN/ERROR, format z datą, dopisywanie, tworzenie katalogu, wyłączenie, awaria zapisu bez wyjątku, 20 równoległych wpisów, ścieżka absolutna
`MainViewModelTests`	9	stan początkowy, wczytanie + drzewo, status z czasem, wybór klasy bez ponownej analizy (kryterium zaliczenia), szczegóły metody, czyszczenie wyboru, błąd + log, `Refresh`, ignorowanie wywołania w trakcie pracy
`TreeNodeViewModelTests`	6	pełna hierarchia, węzły parametrów, szczegóły klasy/parametru/zestawu, powiadomienia o zmianie `IsExpanded` / `IsSelected`

11.4. Przykład testu kryterium zaliczenia

```
 [Fact]
public async Task SelectingNodeUpdatesDetailsWithoutReloading()
{
    _service.Setup(s ⇒ s.AnalyzeAsync(It.IsAny<CancellationToken>
())).ReturnsAsync(assembly);

    var viewModel = CreateViewModel();
    await viewModel.AnalyzeCommand.ExecuteAsync(null);

    viewModel.SelectedNode = viewModel.Assembly!.Nodes[0].Children[0].Children[0];

    Assert.Equal("User", viewModel.Details.Title);
    Assert.Equal("Class", viewModel.Details.Kind);
    _service.Verify(s ⇒ s.AnalyzeAsync(It.IsAny<CancellationToken>()), Times.Once);
}
```

11.5. Scenariusze brzegowe pokryte testami

- pusty model (`AssemblyModel.Empty()`),
- model z jedną przestrzenią nazw,
- model z kilkoma klasami i wieloma metodami,
- niepoprawna operacja (błąd providera, nieznany składnik, pusty argument),
- błąd zapisu logu (plik zamiast katalogu),
- anulowanie operacji,
- równoległe wywołania loggera,
- kultura `pl-PL` vs `InvariantCulture`.

12. Instrukcja obsługi

12.1. Wymagania

- Windows 10/11 (WPF),
- .NET SDK 10.0.x (w tym środowisku: `C:/Users/anysz/.dotnet`, poza `PATH`),

- opcjonalnie Rider / Visual Studio 2026 (obsługują `MetadataAnalyzer.slnx`).

12.2. Build i testy

```
dotnet build MetadataAnalyzer.slnx
dotnet test  MetadataAnalyzer.Tests/MetadataAnalyzer.Tests.csproj
```

Oczekiwany wynik: `Kompilacja powiodła się. 0 błędów, 0 ostrzeżeń`, `powodzenie: 54`.

12.3. Aplikacja WPF

```
dotnet run --project MetadataAnalyzer.WPF
```

1. Kliknij **Analyze** — drzewo pojawi się po lewej, szczegóły zestawu po prawej.
2. Rozwiń `SampleLibrary` → `SampleLibrary.Models` → `User`.
3. Kliknij klasę — panel szczegółów pokaże właściwości, metody, pola i konstruktory.
4. Kliknij metodę lub parametr — panel pokaże podpis, typ zwracany i listę parametrów.
5. **Refresh** wczytuje dane ponownie (osobny wpis w logu).

12.4. Aplikacja CLI

```
dotnet run --project MetadataAnalyzer.CLI
```

Wpisz `1`, potem `2` (drzewo), `3` (podsumowanie), `4` (szczegóły składnika), `0` (wyjście). Zapis przykładowej sesji: `docs/cli-smoke.txt`.

12.5. Log

Plik `Logs/application.log` powstaje obok pliku wykonywalnego:

- WPF: `MetadataAnalyzer.WPF/bin/Debug/net10.0-windows/Logs/application.log`,
- CLI: `MetadataAnalyzer.CLI/bin/Debug/net10.0/Logs/application.log`.

Aby zmienić lokalizację, podaj ścieżkę bezwzględną w `appsettings.json` (`Logging:FilePath`).

13. Decyzje projektowe, clean code, ograniczenia, plan etapu 2 i rozwiązywanie problemów

13.1. Decyzje projektowe i uzasadnienia

Decyzja	Uzasadnienie
`.slnx` zamiast `.sln`	domyślny format nowych rozwiązań .NET 10; rozwiązanie można w razie potrzeby przekonwertować
`Directory.Build.props`	jedno miejsce wspólnych ustawień (`Nullable` , `TreatWarningsAsErrors` , analizatory) dla wszystkich projektów
`record` dla modeli	niemutowalność, równość wartościowa, czytelne `init` -owe kolekcje
`IReadOnlyList<T>` w `Core` , `ObservableCollection` w WPF	model jest danymi, nie UI; obserwacja kolekcji należy do warstwy prezentacji
`ISystemClock`	deterministyczne testy `LoadedAt` i pomiaru czasu operacji
`MetadataProviderResolver`	zachowanie klucza konfiguracji `UseMockData` bez duplikacji kompozycji DI
`SemaphoreSlim` w serwisie i loggerze	brak przeplatania wpisów logu i podwójnej analizy
`DetailsViewModel` jako niemutowalna całość	mniej powiadomień, brak stanów pośrednich, prostsze testy
Formatter poza UI	CLI, logi i przyszły eksport korzystają z jednej implementacji
Brak komentarzy w kodzie	kod opisuje się nazwami; intencja widoczna w sygnaturach i testach
Analizatory jako błędy	wymusza invariantne formatowanie i czystą strukturę od pierwszego commita
Styl ciemny	czytelność przy długiej pracy z kodem; spójna paleta w jednym pliku

Ograniczenie techniczne XAML (świadome obejście): w szablonie `TreeViewItem` nie można użyć `StaticResource` odwołującego się do zasobów zdefiniowanych w `App.xaml` (szablon ładuje się w innym kontekście — błąd `XamlParseException: Cannot find resource named 'BoolToVisibility'`). Rozwiązanie: widoczność przycisku rozwinięcia sterowana triggerem `HasItems` w samym szablonie, a widoczność pustego stanu — kodem w `MainWindow.xaml.cs` reagującym na `HasMetadata` .

13.2. Zgodność z clean code

- **Jedna odpowiedzialność:** model / provider / serwis / formatter / logger / ViewModel / widok.
- **Brak komentarzy** — nazwy metod mówią, co robią (`TryParseIndex` , `DescribeConstructor` , `AppendChildren` , `ResolveLogFilePath`).

- **Brak duplikacji:** `LogEvents` (komunikaty), `DetailsViewModel` (fabryki szczegółów), `ConsolePresentation` (warstwa wizualna CLI), `GetIndent` (wcięcia drzewa).
- **Małe metody**, wcześnie wychodzące (`return` , `continue`), maksymalnie jedna decyzja na metodę.
- **Zabezpieczenia kontraktów:** `ArgumentNullException.ThrowIfNull` , `ArgumentException.ThrowIfNullOrWhiteSpace` , `NotNullWhen(true)` w `TryGetAssembly` .
- **Niezmienność** modeli i `DetailsViewModel` ; brak setterów publicznych tam, gdzie nie są potrzebne.
- **Asynchroniczność** z `CancellationToken` od UI po provider; brak `.Result` / `.Wait()` w kodzie produkcyjnym.
- **Obsługa błędów** na granicy warstw (serwis loguje i propaguje, UI prezentuje, logger nigdy nie rzuca).
- **Formatowanie niezależne od kultury** — wymuszone przez analizator CA1305 (błąd kompilacji).
- **Testy czytelne jak specyfikacja** — nazwy w stylu `AnalyzeFailureShowsErrorAndLogsIt` .
- **Kod ostrzeżeń zero** — brak `#pragma warning disable` , brak `<NoWarn>` .

13.3. Znane ograniczenia etapu 1

1. Dane pochodzą wyłącznie z mocka — brak analizy rzeczywistych plików `.dll` .
2. Logowanie tylko do pliku; logger bazodanowy zarezerwowany na etap 4 (`Logging:Provider`).
3. Brak wyszukiwania i filtrowania w drzewie (etap 2+).
4. Brak eksportu raportu (JSON/CSV) — `IMetadataFormatter` jest na to przygotowany.
5. Ustawienia są tylko do odczytu z `appsettings.json` (bez edycji w UI).
6. Brak testów UI (widoków XAML) — testowane są ViewModele i logika; zgodnie z wymaganiem etapu.
7. `Application:UseMockData = false` rzuca `NotSupportedException` do czasu implementacji refleksji.
8. Aplikacja WPF działa tylko na Windows (WPF nie jest cross-platform).

13.4. Plan etapu 2

1. `ReflectionMetadataProvider : IMetadataProvider` — analiza wskazanego pliku `.dll` przez `System.Reflection` .
2. Uzupełnienie modeli o informacje potrzebne z metadanych (atrybuty, typy ogólne, interfejsy, dziedziczenie).
3. Okno wyboru pliku w WPF i parametr ścieżki w CLI.
4. Filtrowanie i wyszukiwanie w drzewie.

5. Rozszerzenie testów o próbkę zestawu testowego (`TestAssembly.dll`) i scenariusze błędnych plików.

13.5. Rozwiązywanie problemów

Objaw	Przyczyna	Rozwiązanie
`dotnet: command not found`	SDK nie jest w `PATH`	dodaj `C:/Users/anysz/.dotnet` do `PATH` lub używaj pełnej ścieżki
`You must install .NET to run this application`	uruchamianie `.exe` bez `dotnet` w `PATH`	uruchamiaj przez `dotnet run --project …`
Brak pliku `Logs/application.log`	`Logging:Enabled = false` lub brak praw zapisu	ustaw `Enabled: true` , sprawdź uprawnienia katalogu wyjściowego
Log zawiera `[logging failure] …`	plik/katalog zablokowany lub ścieżka wskazuje plik	popraw `Logging:FilePath`
Okno WPF puste po `Analyze`	błąd analizy — sprawdź pasek statusu i log	przeczytaj wpis `ERROR` w logu
`XamlParseException` przy starcie	brak zasobu w kontekście szablonu	nie wiąż zasobów aplikacji w `ControlTemplate` ; użyj triggerów
CLI kończy się od razu	zamknięty strumień wejścia (przekierowanie)	uruchom interaktywnie w konsoli

13.6. Historia wersji dokumentu

Wersja	Data	Zmiany
1.0	2026-11-10	pierwsze wydanie
1.1	2026-11-11	usunięcie wersji online, ujednolicenie nazwy `SampleLibrary.dll` , poprawki spisu treści